Université du Québec
à Chicoutimi

Département d'informatique et mathématique

PROGRAMMATION ORIENTÉE OBJET

8PRO128

Projet : Gestion d'une agence immobilière

Hiver 2026

Groupe 01

Chargée de cours : Asma Zouaghi

Courriel : azouaghi@uqac.ca

Développement d'une Application pour Agence Immobilière

Sujet : Une agence immobilière vous demande de développer une application C++. Cette agence met en relation les personnes qui sont à la recherche d'un logement à acheter ou à louer et celles qui en ont de disponibles soit à la vente soit à la location. L'agence peut aussi gérer les problèmes quotidiens (des réclamations / des suggestions) des propriétaires et des locataires en leur servant d'intermédiaire. Dans le cadre de la gestion locative, la mission de l'agence est de : rechercher un locataire, rechercher un logement, établir les contrats, les quittances de loyer, etc. En contre partie du temps gagné par les clients, pour une vente ou une location d'un bien immobilier (maison, appartement, terrain, etc.), l'agence perçoit un pourcentage sur la valeur du bien ou du loyer à la signature de l'acte notarié ou du contrat de location. L'application devra posséder au minimum les fonctionnalités décrites ci-dessous.

<u>Vous devez ajouter les fonctionnalités que vous jugez nécessaires.</u>

Objectif : L'objectif de ce projet est de développer une application permettant de mettre en application toutes les notions que vous êtes en train de voir durant les séances de cours/TDs de POO (appliquée au langage C++). Vous êtes invité à utiliser toutes les notions vues à savoir : les classes, les objets, les constructeurs, les destructeurs, l'héritage, le polymorphisme, les templates, les exceptions. Il est crucial de tester régulièrement le code pour s'assurer de son bon fonctionnement. En complétant ces exercices, vous aurez l'occasion de mettre en pratique tous les concepts clés de la programmation orientée objet en C++.

Partie 1 : Modélisation UML

Diagramme de Classes : les classes principales et leurs relations peuvent être définies comme suit :

Classes principales :

1. **BienImmobilier** : classe de base pour les biens.
   Attributs : id, adresse, surface, type, statut.
   Méthodes : afficherDetails(), mettreAJourStatut(string nouveauStatut).
   Sous-classes : Maison, Appartement, Terrain.
2. **Personne** : classe de base pour les individus.
   Attributs : nom, adresse, telephone.
   Méthodes : afficherInfos().
   Sous-classes : Client, Proprietaire, Locataire.

3. **Contrat** : classe pour la gestion des contrats de vente ou de location.
   Attributs : idContrat, date, typeContrat, termesContrat.
   Méthodes : afficherContrat(), signerContrat().
4. **Transaction** : classe pour représenter les transactions financières.
   Attributs : idTransaction, montant, dateTransaction.
   Méthodes : effectuerTransaction().
5. **Agence** : une classe qui permet de relier les clients et les biens immobiliers.
   Méthodes :ajouterBien(),ajouterClient(),creerContrat(),enregistrerTransaction()

Associations :

- Un Client peut avoir plusieurs Contrats.
- BienImmobilier est associé à Contrat pour spécifier quel bien est concerné.
- La classe Agence sert d'intermédiaire entre Client et BienImmobilier.

Considérons le processus de location d'un appartement :

- Le Client recherche un Appartement à louer.
- Agence fournit les options disponibles.
- Le Client choisit un Appartement.
- Agence crée un Contrat de location.
- Client et Proprietaire signent le Contrat.
- Une Transaction est enregistrée pour le paiement du loyer.

Partie 2 : Implémentation en C++

Exercice 1 : Classes de base et objets

Objectif : comprendre et mettre en pratique la création de classes de base, la déclaration d'attributs, l'implémentation de méthodes, et l'instanciation d'objets.

1. Définir la classe *BienImmobilier* avec ses attributs et méthodes comme indiqué.
2. Implémenter la méthode afficherDetails() pour afficher les informations du bien.
3. Créer les classes *Personne* et *Contrat*, en définissant leurs attributs et méthodes.
4. Instancier plusieurs objets de ces classes dans la fonction *main()* et appeler leurs méthodes pour tester leur bon fonctionnement.

Exercice 2 : Héritage

Objectif : appliquer le concept d'héritage pour créer des sous-classes spécifiques à partir de classes de base.

1. Créer les sous-classes *Maison*, *Appartement* et *Terrain* qui héritent de *BienImmobilier*. Ajouter les attributs et méthodes spécifiques si nécessaire.
2. Définir les sous-classes *Client*, *Proprietaire*, et *Locataire* qui héritent de *Personne*.
3. Tester l'héritage en instanciant des objets des sous-classes dans la fonction *main()* et en appelant leurs méthodes ainsi que celles héritées de la classe de base.

Exercice 3 : Polymorphisme

Objectif : Exploiter le polymorphisme pour permettre à des méthodes de se comporter différemment en fonction de l'objet qui les appelle.

1. Implémenter une méthode virtuelle dans la classe *BienImmobilier* et redéfinisser-la dans chaque sous-classe pour afficher les détails spécifiques à chaque type de bien.
2. Tester le polymorphisme dans la fontion *main()* en utilisant un pointeur de type *BienImmobilier* pour appeler cette méthode sur différents types de biens et observer le polymorphisme en action.

Exercice 4 : Gestion des exceptions

Objectif : Utiliser les exceptions pour gérer les erreurs et les cas particuliers de manière sécurisée.

1. Implémenter un mécanisme de gestion des exceptions dans les méthodes critiques, comme la création de contrats ou la réalisation de transactions, pour gérer par exemple des entrées invalides ou des erreurs de traitement.
2. Utiliser des blocs try/catch pour capturer et gérer ces exceptions.
3. Tester le mécanisme en provoquant délibérément des erreurs et en vérifiant que les exceptions sont bien capturées et gérées.

Exercice 5 : Patrons de fonctions et de classes

Objectif : Démontrer l'utilisation des *templates* pour créer des fonctions ou des classes génériques capables de travailler avec différents types de données.

1. Créer un *template* de fonction ou de classe qui pourrait être utile dans le contexte de votre application, comme une fonction de comparaison pour trier des biens immobiliers ou des clients selon différents critères.
2. Tester votre template avec différents types pour vérifier sa flexibilité.

Exercice 6 : Association et Aggrégation

Objectif : Modéliser des associations et aggrégations entre classes pour refléter les relations complexes entre les entités.

1. Implémenter des relations d'association entre Client et Contrat, ainsi qu'entre BienImmobilier et Contrat, en utilisant des vecteurs pour gérer les multiples associations.
2. Ajouter une composition ou aggrégation dans la classe Agence pour gérer le vecteur de BienImmobilier et de Personne.

Modalités de travail en groupe : Ce projet est à réaliser en groupe de travail composé de maximum 4 étudiants.

Le rendu : le rendu de votre projet devrait se composer de deux parties principales : le code et un rapport écrit. Voici les éléments clés à inclure :

1. **Code :** structuré avec des fichiers séparés pour chaque classe ou fonctionnalité majeure. Utiliser des commentaires pour expliquer la logique, surtout pour les parties complexes.
2. **Rapport :**
    a. détails par exercice : pour chaque exercice, expliquer la conception, l'implémentation, et comment cela s'intègre dans l'application globale ;
    b. défis et solutions : discuter des problèmes rencontrés et comment vous les avez résolus ;
    c. tests et validation : décrire les tests réalisés pour chaque partie du code et présenter les résultats pour confirmer le bon fonctionnement de l'application ;
    d. réflexions et améliorations futures : résumer ce que vous avez appris et envisager des améliorations ou des extensions possibles.

Date de rendu : Le 29 avril 2026 – avant 23h59.

Bonne chance ☺ !